

Homework 1

In this assignment, as with all other work in our class, you are required to work in teams. Before you begin the assignment, read through the course syllabus.

The primary goal of this assignment is to demonstrate two key ideas for this class:

- 1. You are comfortable structuring a Python project from broad instructions.**
- 2. You can synthesize fragments of code from our lectures and labs into a thoughtful solution to a problem.**

It's crucial to break away from the traditional Jupyter Notebook mold that leaves many data scientists thinking Python software development begins and ends inside of a Jupyter Notebook. In practice, Python software that is shared across teams, organizations, or other groups is typically done using project structures. That said, Jupyter Notebooks are useful for pedagogy and prototyping code, and that's why our lectures will sometimes adopt this format. **Your team's submission must be a Python project following modern project structuring** described in our Modern Python Packaging lab: a `pyproject.toml`, never `setup.py`.

A major part of this course is learning to be adaptable: expecting new and unfamiliar problems, and becoming comfortable being uncomfortable. To that end, our assignments are intentionally organized in an unstructured way, through broadly-defined **Task Lists**. Rather than spelling out every deliverable, the task lists describe what is expected and leave some decisions up to you. How exciting.

What you are building

Over the semester, your team will build a production-grade, cloud-deployed application of your own, one week at a time. You choose the application. It should be something an ML model sits inside of and that a real user would call: a classifier behind an API, a search or retrieval service, a summarizer, a recommender system, an image or audio service, or something else of your choice.

This first assignment builds the installable foundation of that project: a clean, packaged, tested, CI-checked service skeleton. The model itself does not have to do anything interesting yet; this week is about the software around the model, not the model or whatever project you and your team ultimately choose to work on.

Problem 1.

In this class, you will be submitting your work using Git. So, to get started, you should make an account on GitHub.

Make a [GitHub](#) account if you do not have one. GitHub is the de facto choice for version control and makes team collaboration seamless. If you already feel comfortable with Git, skip the tutorials; otherwise work through one:

- <https://docs.github.com/en/get-started/quickstart/hello-world>
- <https://product.hubspot.com/blog/git-and-github-tutorial-for-beginners>
- <https://www.dataschool.io/git-and-github-videos-for-beginners/>

Another typical Git server is called GitLab, which you may see in the workplace.

Problem 2.

Throughout this class, we'll use a compute cluster provided by the University of Indiana called Jetstream2. There is extensive documentation for this cloud computing cluster available [here](#). For your convenience, you can review the most important points so you can get started with your team by watching [this playlist](#) of two videos generously provided by Jetstream2 Team. The first video is about 30 minutes, and the second video is about one and a half hours. You should carefully review both of these videos.

Once you've watched the videos, follow the [instructions here](#) to create an ACCESS ID so that you can be added to the Jetstream2 allocation for our class.

Send just one email to the TA team and instructor when every member of your team has ACCESS IDs. Then, your team will be added to the allocation.

Problem 3.

This is the heart of the assignment. You will apply modern Python development tools to build an installable, tested, CI-checked package that comes up healthy on any machine. The underlying task your team solves is unimportant and the modeling approach you take for the task can thus be treated as a black box.

Software Task List

- ☐ Make sure your team shares one environment (via uv.lock) and that everyone has the repository cloned with remote tracking set up. For now it is enough to share the repo and push to it.
- ☐ Scaffold a package with the src/ layout. Use uv init (you may start from the course cookiecutter template) to create a project whose package lives under src/.
- ☐ Write your pyproject.toml. Declare a build backend (e.g. hatchling), your runtime dependencies, and a dev optional-dependency group for your tooling (ruff, pytest, mypy, pre-commit). Commit the generated uv.lock.
- ☐ Stand up a small FastAPI service skeleton. Your app does not have to do anything interesting yet, but it must be a real web service:
 - ☐ a FastAPI application with at least a /health endpoint that returns a small typed response and lets a caller (or a container, or a grader) confirm the service is up;
 - ☐ at least one Pydantic request/response model so input is validated at the edge and output is serialized predictably;
 - ☐ a placeholder "predict"-style endpoint is welcome but optional this week.
- ☐ Configure your application. Add typed configuration with Pydantic Settings: the app reads its settings from the environment, with validators that fail fast at startup on bad config (crash loudly, do not limp along). Provide paired example environment files (for example a local-dev shape and a deployed shape) so a new developer knows exactly which knobs exist. Never commit real secrets.
- ☐ Add quality checks:
 - ☐ ruff for linting and formatting (run both)
 - ☐ pytest with at least one meaningful test — for example, hitting your /health endpoint with FastAPI's test client, or asserting that your settings validation rejects a bad value. A test that asserts nothing does not count
 - ☐ pre-commit with a Git hook that runs ruff (and whatever else you like) before each commit, so low-quality code never enters the repository
- ☐ Incorporate GitHub Workflows. Add a GitHub Actions workflow at .github/workflows/ci.yml that runs on every push and pull request and performs each step:
 - ☐ pick an operating system and install Python;
 - ☐ check out the repository;
 - ☐ install uv and run uv sync --extra dev --frozen;
 - ☐ run uv run ruff check and uv run ruff format --check;
 - ☐ run uv run mypy src/ (type-checking);
 - ☐ run uv run pytest
- ☐ Provide documentation. Write an informative README with clear, copy-pasteable instructions for installing and running your service on a clean machine. The [Diátaxis](#) framework offers sound advice on structuring technical documentation. Add docstrings to your functions (autoDocstring can scaffold them).

Grading

- ☐ **Your documentation in your README files matters the most: it is important to be as clear as possible on how to run your code.** The best way to ensure your code runs on another machine is by doing a fresh installation of your environment to guarantee there won't be any environment issues.

- ❑ **In this assignment, your goal is to be a good member of a team working together to share code using best practices. The instruction team is looking for good Python project structuring and CI implementation using GitHub Actions.**

Grading Guide used by the TA Team.

- ❑ Problems 1 and 2: Students should provide their ACCESS IDs in a single email per group (2 points).
- ❑ Problem 3: Students should have evidence of a working inference API.
 - ❑ Packaging. src/ layout, a valid pyproject.toml with a build backend and a dev dependency group, and a committed uv.lock. (2 points)
 - ❑ Installs on a clean machine. A fresh uv sync --frozen installs the project from the lockfile with no manual changes needed. (2 points)
 - ❑ FastAPI skeleton comes up healthy. The service starts and a /health endpoint returns a valid typed response; at least one Pydantic model is used. (2 points)
 - ❑ Configuration. Typed Pydantic Settings read from the environment. (2 points)
 - ❑ Quality gates. ruff clean, a pre-commit hook, and at least one meaningful pytest test. (2 points)
 - ❑ A GitHub Actions workflow runs the gates on push, and it passes. (2 points)
 - ❑ Documentation. A clear README with run instructions, plus docstrings on your functions. (1 point)